25. Graphs: BFS, DFS, Shortest Paths Intro

25. Graphs: BFS, DFS, Shortest Paths Intro

Phase: Phase 4 — Algorithms and data structures
Estimated time: 2 days
Slug: `25-graphs`

Learning objectives

- Represent graphs with adjacency lists
- Implement BFS and DFS
- Detect cycles in directed graphs
- Introduce Dijkstra at a conceptual+code level

Prerequisites

Complete prior numbered lessons. Type every example yourself — reading Go is not writing Go.

Why this matters

Graphs model networks, dependencies, and workflows. BFS/DFS are the default tools for reachability and shortest unweighted paths.

This lesson is a building block for later modules in the Goodway path. If you only skim, the exercises will feel like busywork; if you type the examples and narrate them aloud, the same patterns will reappear in HTTP handlers, concurrency, and AI service glue.

Core concepts

- Represent graphs with adjacency lists: map[T][]T.
- BFS uses a queue; DFS uses stack/recursion.
- Mark visited to avoid cycles.
- Unweighted shortest path = BFS layers.
- Directed vs undirected changes edge insertion.

Hold these in working memory while you code. Each bullet should map to something you can demonstrate with a three-line program or a failing compile error.

Worked Go examples

Create a scratch module and type this yourself (do not paste blindly):

```
mkdir -p /tmp/gw-graphs: && cd /tmp/gw-graphs:
go mod init example.com/scratch
```

Then create `main.go`:

```
package main
```

```
import "fmt"

func BFS(start string, adj map[string][]string) []string {
    seen := map[string]bool{start: true}
    q := []string{start}
    var order []string
    for len(q) > 0 {
        u := q[0]
        q = q[1:]
        order = append(order, u)
        for _, v := range adj[u] {
            if !seen[v] {
                seen[v] = true
                q = append(q, v)
            }
        }
    }
    return order
}

func main() {
    adj := map[string][]string{"A": {"B", "C"}, "B": {"D"}, "C": {}, "D": {}}
    fmt.Println(BFS("A", adj))
}
```

Run it:

```
go run .
```

Expected output:

```
[A B C D]
```

If your output differs, diff character-by-character — especially whitespace and types. Then change one line deliberately and predict the new output before running.

Mini-lab (`go run` steps)

1. BFS a small graph; print visit order.
2. DFS recursive and iterative.
3. Compute unweighted distance via BFS layers.

Write down one sentence explaining *why* the output matches the code before you move on. Time-box the lab to 15–25 minutes; struggle is part of encoding the skill.

Curriculum notes (expanded from the compact draft)

Representation

```
type Graph map[string][]string // adjacency list
```

Prefer adjacency lists for sparse graphs (most real networks).

BFS / DFS

- BFS: queue — shortest path in unweighted graphs
- DFS: stack/recursion — cycle detection, topological vibes

```go
func BFS(g Graph, start string) []string {
    seen := map[string]bool{start: true}
    q := []string{start}
    var order []string
    for len(q) > 0 {
        u := q[0]; q = q[1:]
        order = append(order, u)
        for _, v := range g[u] {
            if !seen[v] { seen[v] = true; q = append(q, v) }
        }
    }
    return order
}
```

Weighted shortest paths

Dijkstra with a priority queue finds shortest paths for non-negative weights — core infra for routing and dependency costs.

Failure modes to watch while coding

- Adjacency matrix for huge sparse graphs
- Forgetting seen set → infinite loops
- Dijkstra with negative edges

When a test fails, ask which failure mode you hit before changing code at random.

Idioms vs Python pitfalls

- Same algorithms; Go maps+slices are the usual adjacency structure.
- collections.deque vs slice queue (O(n) head pop — ok for learning).
- NetworkX hides representation — implement by hand here.

When you catch yourself reaching for a Python habit, pause and rewrite the Go way once on purpose — even if both would “work”. The goal is a Go-shaped intuition.

Going deeper

For weighted shortest paths, Dijkstra comes next — know when BFS is enough.

Practice checklist

Before the exercise, confirm you can:

1. Explain “Graphs: BFS, DFS, Shortest Paths Intro” aloud in under two minutes without notes.
2. Re-type the worked example from memory and match the expected output.

3. Name the Go standard library packages touched by this lesson.
4. List one production bug this lesson's rules prevent.
5. Contrast the Go approach with the Python instinct listed above.
6. Write a three-line cheat sheet: one idiom, one pitfall, one command to remember.

If any item fails, re-run the mini-lab once more, then attempt the exercise tests.

Transfer to production

Ask where this shows up outside toys: config loading, API handlers, data pipelines, CI scripts, or on-call debugging. Write one concrete two-sentence scenario tying the worked example to that scenario. Tomorrow, when you rewrite the exercise from memory, check whether the code still fits that framing.

Common mistakes (especially from Python instincts)

- Adjacency matrix for huge sparse graphs
- Forgetting seen set → infinite loops
- Dijkstra with negative edges

Exercises

Work in `exercises/25-graphs/`. Read that folder's `README.md` before you code.

What you will build

Implement BFS (or the exercise's graph walk) to match expected visit order.

Study loop

- **A (Drill):** make unit tests pass with the simplest correct code.
- **B (Apply):** extend with an edge case called out in the exercise README.
- **C (Challenge):** rewrite your solution from memory the next day without looking.

Do not open `solutions/25-graphs/` until you have an honest failing attempt.

Self-check

Answer without scrolling up:

1. When BFS vs DFS?
2. Why adjacency list?
3. Dijkstra precondition?

Further reading

- https://en.wikipedia.org/wiki/Graph_traversal